Aprendizado de Máquina - Árvore de Decisão: Teoria

IFSP - PEP

Cláudio M. Zaina

Índice

Árvore de Decisão **2**
Medidas de escolha da condição de teste de atributo . . . 3
Impureza . . . 3
Bibliotecas . . . 6
Algoritmo para construção de uma Árvore de Decisão . . . 8
Impureza coletiva dos nós-filhos . . . 8
Identificação da variável . . . 8
Estado civil (`civil`) . . . 8
`sexo` . . . 9
Nível Escolar (`educacao`) . . . 9
`credito` . . . 11
`salario` . . . 13
Escolha da decisão . . . 13
Dados de treino, teste e validação . . . 19
Modelo de Árvore de Decisão . . . 20
Modelo de Árvore de Decisão Podada . . . 23
Conjunto de modelos com `caret::train` . . . 26
Caso 2 - Fraude no Seguro . . . 27
Dados . . . 27
Modelo . . . 28
Modelo com peso . . . 29
Modelo com custo . . . 30
Modelo com fator . . . 31
Modelo com poda . . . 32
Caso 3 - Banco . . . 33
Dados originais . . . 34
Dados processados . . . 35
Separação dos conjuntos de *treino*, *validação* e *teste* . . . 35
Dados separados em *treino*, *validação* e *teste* . . . 36
Modelo base de comparação (*baseline*) . . . 36
Modelo com ponderação . . . 37
Validação . . . 39
Teste . . . 39

Árvore de Decisão

Árvore de Decisão é uma técnica simples de classificador, bastante útil quando as relações que definem a classe da variável resposta são não-lineares ou quando é importante para a classificação interações entre as variáveis independentes (*features*).

Suas qualidades são:

- não é necessário padronização das variáveis pois apenas a posição do valor (posto ou *rank*) é importante;
- é robusta para valores perdidos (`NA`s);
- o modelo é fácil de interpretar;
- variáveis (atributos ou *features*) pouco importantes são automaticamente dispensadas;
- indicadores de impureza são consistentes entre si, fazendo com que todos sejam coerentes com seus resultados;
- seu desempenho é comparável àqueles das demais técnicas.

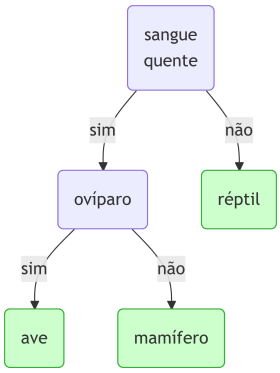
As limitações mais imediatas da técnica da árvore de decisão são:

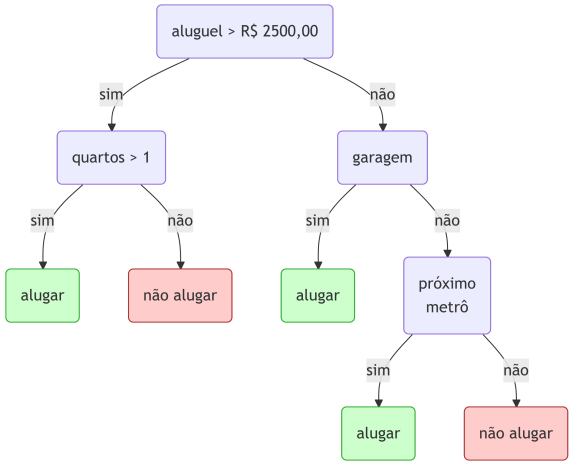
- uma variável categórica com muitos tipos diferentes de resposta (`profissão`, `nacionalidade`, `fabricante`, `sintoma` etc) pode, para árvores não binárias, criar classificações muito específicas para o conjunto de dados;
- tem propensão a superajuste (*overfitting*);
- insensível para variáveis com interação significativa. Se variáveis só são importantes para a definição da classe quando *juntas*, o método guloso de construção da árvore vai decidir por cada uma em separado, construindo árvores complexas demais, desperdiçando a informação que elas só fornecem *juntas*. Por exemplo, Índice de Massa Corpórea e circunferência abdominal. Cada indicador isolado é inconclusivo; juntos indicam risco real

A árvore é composta de três tipos de nós (vértices):

- **raiz**: um nó sem anteriores, ligado a nenhum ou mais (normalmente 2, no nosso caso) nós;
- **nó interno**: nó que não é raiz nem folha; possui só um nó anterior e 2 (ou mais) nós seguintes;
- **folha**: nó que não possui nós seguintes; é o resultado da classificação.

Todo nó não-folha contém um teste de condição sobre (geralmente) um atributo. O resultado desse teste tem uma saída para cada resultado do teste. Uma vez atingido uma folha, uma classe é atribuída a ela. Exemplos podem ser observados a seguir, para determinar (grossamente) a classe de um animal ou se um imóvel é um bom negócio para ser alugado ou não:





Medidas de escolha da condição de teste de atributo

Há diversas medidas que podem ser usadas para a adequação da condição de teste de um atributo. O objetivo é obter um teste que obtenha a separação mais *limpa* ou *pura* dentre as classes nos nós filhos. Nós *puros* não precisam mais ser divididos.

Impureza

A impureza de um nó mede quão diferentes são os elementos (segundo a classificação desejada) que "caíram" naquele nó após a separação (do nó anterior). Há diversas medidas para avaliar a impureza de um nó t, sendo c o número de classes e $p_i(t)$ a frequência relativa das instâncias da classe i no nó t.

$$\text{Entropia} = -\sum_{i=0}^{c-1} p_i(t) \cdot log_2(p_i(t))$$

$$\text{Índice Gini} = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

$$\text{Erro de Classificação} = 1 - \max_i[p_i(t)]$$

Observação: consideramos $0 \cdot log_2(0) = 0$.

Vamos supor que tenhamos 4 nós N_1, N_2, N_3 e N_4, cada um com uma quantidade diferente de elementos das classes `0` e `1` mas separando seis observações do nó anterior:

Sendo c o número de classes e $p_i(t)$ a frequência relativa das instâncias da classe i no nó t.

Tabela 1: Nó com 0 `0`s, 6 `1`s

N1		quantidade
classe	`0`	0
	`1`	6

$$\begin{aligned}
\text{Entropia} = & \quad -(\tfrac{0}{6} \cdot log_2(\tfrac{0}{6}) + \tfrac{6}{6} \cdot log_2(\tfrac{6}{6})) & = 0 \\
\text{Gini} = & \quad 1 - ((\tfrac{0}{6})^2 + (\tfrac{6}{6})^2)) & = 0 \\
\text{Erro clasificação} = & \quad 1 - max(\tfrac{0}{6}, \tfrac{6}{6}) & = 0
\end{aligned}$$

As observações foram totalmente separadas e, neste nó, só há uma classe. A impureza portanto é zero.

Tabela 2: Nó com 1 `0`, 5 `1`s

N2		quantidade
classe	`0`	1
	`1`	5

$$\begin{aligned}
\text{Entropia} = & \quad -(\tfrac{1}{6} \cdot log_2(\tfrac{1}{6}) + \tfrac{5}{6} \cdot log_2(\tfrac{5}{6})) & = 0{,}650 \\
\text{Gini} = & \quad 1 - ((\tfrac{1}{6})^2 + (\tfrac{5}{6})^2)) & = 0{,}278 \\
\text{Erro clasificação} = & \quad 1 - max(\tfrac{1}{6}, \tfrac{5}{6}) & = 0{,}167
\end{aligned}$$

O valor da impureza sobe rápido. Basta uma observação em seis ser de uma classe diferente que a impureza vai para 0,65, no caso da entropia.

Tabela 3: Nó com `0`:2, `1`:4

	N3	quantidade
classe	`0`	2
	`1`	4

$$\begin{aligned}
\text{Entropia} = & \quad -(\tfrac{2}{6} \cdot log_2(\tfrac{2}{6}) + \tfrac{4}{6} \cdot log_2(\tfrac{4}{6})) & = 0{,}918 \\
\text{Gini} = & \quad 1 - ((\tfrac{2}{6})^2 + (\tfrac{4}{6})^2)) & = 0{,}444 \\
\text{Erro clasificação} = & \quad 1 - max(\tfrac{2}{6}, \tfrac{4}{6}) & = 0{,}333
\end{aligned}$$

Com uma observação em três sendo de uma classe diferente, a entropia, por exemplo, se aproxima do máximo.

Tabela 4: Nó com `0`:3, `1`:3

	N4	quantidade
classe	`0`	3
	`1`	3

$$\text{Entropia} = \quad -(\tfrac{3}{6} \cdot log_2(\tfrac{3}{6}) + \tfrac{3}{6} \cdot log_2(\tfrac{3}{6})) \quad = 1$$

$$\text{Gini} = \quad 1 - ((\tfrac{3}{6})^2 + (\tfrac{3}{6})^2)) \quad = 0{,}5$$

$$\text{Erro clasificação} = \quad 1 - max(\tfrac{3}{6}, \tfrac{3}{6}) \quad = 0{,}5$$

Quando, após a separação, a quantidade de elementos de uma classe é igual àquela da outra, a entropia é máxima em 1 (a impureza é total).

Funções para o cálculo da `entropia`, índice de Gini (`gini`) e erro de classificação (`erro_clas`) para casos com apenas 2 classes:

```
rm(list=ls())
```

```
entropia <- function(a0,a1) { # qtd de categoria 0, qtd de categoria 1
    n <- a0 + a1
    if (n == 0) { return (0) }
    p1 <- a0/n
    p2 <- a1/n
    if (p1 == 0) {
        k1 <- 0
    } else {
        k1 <- p1 * log2(p1)
    }
    if (p2 == 0) {
        k2 <- 0
    } else {
        k2 <- p2 * log2(p2)
    }
    return (- (k1 + k2))
}
gini <- function(a0,a1) {
    n <- a0 + a1
    if (n == 0) {
        return (0)
    } else {
        return (1 - a0/n * a0/n - a1/n * a1/n)
    }
}
erro_clas <- function(a0,a1) {
    n <- a0 + a1
    if (n == 0) {
        return (0)
    } else {
        return (1 - max(a0/n, a1/n))
    }
}
```

Exemplo dos valores para 1 em 6:

```r
a0 <- 1
a1 <- 5
print(paste("Entropia:  ",round(entropia(a0,a1), 4)))
print(paste("Gini:      ",round(gini(a0,a1), 4)))
print(paste("Erro clas.:",round(erro_clas(a0,a1), 4)))
```

```
[1] "Entropia:   0.65"
[1] "Gini:       0.2778"
[1] "Erro clas.: 0.1667"
```

Bibliotecas

```r
library(rpart)         # árvores de decisão
library(rpart.plot)    # gráficos de árvores
library(readr)         # leitura de csv
library(repr)          # representações (ajuste gráfico do jupyter)
library(caret)         # matriz de confusão
library(caTools)       # ferramentas em geral; no caso função que divide datasets
library(RColorBrewer)  # gradientes de cores
```

```r
grafdim <- function(x=11, y=9) { options(repr.plot.width=x, repr.plot.height=y) }
grafdim()
```

Vamos construir um conjunto de dados sintético (artificial) com regras claras e ver se a técnica da Árvore de Decisão é capaz de recuperá-las.

```r
set.seed(997)
```

```r
criar_emprestimo <- function(tamanho, semente=997) {
    if (!is.na(semente)) {
        set.seed(semente)
    }
    empr <- data.frame(
        # 30% de solteiros, 70% de casados
        civil    = factor(sample(c('solteiro','casado'), tamanho,
                                 replace=TRUE, prob=c(0.3, 0.7))),
        # 50% para cada sexo
        sexo     = factor(sample(c('M','F'), tamanho,
                                 replace=TRUE, prob=c(0.5, 0.5))),
        # 20% para ensino médio, 60% para nível superior e 20% para pós
        educacao = factor(sample(c("médio", "superior", "pós-graduação"),
                                 tamanho, replace=TRUE,
                                 prob=c(0.2, 0.6, 0.2)),
                                 ordered=TRUE),
        # crédito tem distribuição uniforme em [0,55; 0,95]
        credito  = round(runif(tamanho, 0.55, 0.95) * 10, 1),
        # salário tem discribuição Wibull com parâmetros 1,6 e 2
        # para uma forma com mais densidade em salários mais baixos
        salario  = round(rweibull(tamanho,1.6,2)*40,4)*100 + 5000
    )
    # as regras "secretas" que motivam a concessão de um empréstimo
    empr$emprest <- factor(as.numeric(
        empr$civil == 'solteiro' & (empr$sexo == 'F' & empr$credito > 7.5 &
```

```
                                              empr$salario > 15000 |
                                    empr$sexo == 'M' & empr$credito > 9.0) |
        empr$salario > 25000 & empr$credito > 0.7 |
        empr$educacao == 'superior' & empr$salario > 10000 &
                         empr$credito > 0.6 & empr$civil == 'casado' |
        empr$educacao == 'pós-graduação' & empr$credito > 7.0 & empr$salario > 7000
    ))
    return (empr)
}
```

```
empr <- criar_emprestimo(10)
```

```
summary(empr)
```

```
      civil    sexo           educacao    credito          salario       emprest
 casado  :6   F:4   médio        :3   Min.   :5.600   Min.   : 7416   0:6
 solteiro:4   M:6   pós-graduação:1   1st Qu.:6.975   1st Qu.: 8726   1:4
                    superior     :6   Median :7.700   Median :10891
                                      Mean   :7.660   Mean   :13072
                                      3rd Qu.:8.350   3rd Qu.:15575
                                      Max.   :9.500   Max.   :29346
```

Se a semente for 997 e `tamanho` for 10, os dados gerados serão:

Tabela 5: Tabela com dados para exemplo

civil	sexo	educacao	credito	salario	emprest
casado	F	pós-graduação	9,5	17479,44	1
casado	M	superior	9,3	10546,73	1
solteiro	F	superior	7,8	29345,74	1
casado	F	médio	6,3	12428,51	0
solteiro	M	médio	8,5	9561,46	0
solteiro	M	superior	6,9	16623,75	0
casado	F	médio	7,2	7415,93	0
casado	M	superior	7,6	8446,85	0
casado	M	superior	5,6	11234,57	1
solteiro	M	superior	7,9	7638,89	0

como resultado de:

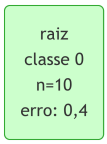
```
exemplo <- head(empr, 10)
print(exemplo)
```

```
      civil sexo      educacao credito  salario emprest
1    casado    F pós-graduação     9.5 17479.44       1
2    casado    M      superior     9.3 10546.73       1
3  solteiro    F      superior     7.8 29345.74       1
4    casado    F         médio     6.3 12428.51       0
5  solteiro    M         médio     8.5  9561.46       0
6  solteiro    M      superior     6.9 16623.75       0
7    casado    F         médio     7.2  7415.93       0
8    casado    M      superior     7.6  8446.85       0
9    casado    M      superior     5.6 11234.57       1
10 solteiro    M      superior     7.9  7638.89       0
```

Algoritmo para construção de uma Árvore de Decisão

Muitas árvores de decisão podem ser construídas para um conjunto de dados. Decidir qual a melhor é computacionalmente intensivo, devido à quantidade de decisões possíveis. A abordagem utilizada é *gulosa* (*greedy* - gananciosa) e localmente ótima sobre qual atributos utilizar para decidir o próximo nível.

A base para os algoritmos atuais é o Algoritmo de Hunt. O início é com um nó (raiz) contendo todas as observações e atribuído à classe mais frequente. Assim, para o nosso caso, teremos um primeiro nó, com 10 observações, classificado como `0` (caso *negativo*, não concede empréstimo). O erro desse nó é 0,4, uma vez que há 4 casos (em 10) positivos classificados como negativos. Esse valor de erro é importante, pois normalmente é utilizado como critério de parada.



Uma vez que ainda há elementos de ambas classes no nó, a árvore pode ser expandida pela aplicação de uma decisão sobre um dos seus atributos. A pergunta então é qual atributo e qual valor?

O foco aqui será para a construção de árvores de decisão binárias.

Impureza coletiva dos nós-filhos

Considere uma critério de decisão d que divida um nó w com N observações em k nós-filhos $\{v_1, v_2, v_3, ..., v_k\}$. Vamos focar em árvores binárias, nas quais $k = 2$, com apenas $\{v_1, v_2\}$ mas a teoria é genérica. Vamos chamar de $N(v_i)$ a quantidade de observações que restaram no nó v_i após a divisão. Podemos calcular a impureza $I(v_i)$ desse nó. A impureza ponderada dos filhos de um nó w seria:

$$I(w_d) = \sum_{i=1}^{k} \frac{N(v_i)}{N} I(v_i)$$

```
impcol <- function(a0,a1,b0,b1,funcao=gini) {  # baseada na entropia
    a <- a0 + a1
    b <- b0 + b1
    n <- a + b
    if (a == 0) { ea <- 0 } else { ea <- funcao(a0,a1) }
    if (b == 0) { eb <- 0 } else { eb <- funcao(b0,b1) }
    ic <- (a/n) * ea + (b/n) * eb
    return (ic)
}
```

Identificação da variável

Qualquer das métricas (Entropia, Gini ou Erro de Classificação) pode ser utilizada para calcular a impureza dos nós-filhos. O Índice de Gini é o mais comum pelo seu baixo custo computacional e é, portanto, padrão nas diversas bibliotecas que modelam árvores de decisão.
Vamos verificar a impureza coletiva do nó inicial segundo seus atributos pela entropia.

Estado civil (`civil`)

Calculando a impureza coletiva do nó $w = raiz$ para o estado civil.
Como curiosidade, a impureza coletiva para `civil` será calculada com entropia e Gini. As demais variáveis apenas com Gini. Observando o exemplo, Tabela 5 com as 10 observações, há 4 casos com `civil` = `solteiro` e apenas 1 deles conseguiu empréstimo.
Há 6 casos com casados, sendo que 3 deles conseguiram empréstimo:

Entropia

$$
\begin{aligned}
I(raiz_{\text{civil=solteiro}}) = & \quad -(\tfrac{3}{4} \cdot log_2(\tfrac{3}{4}) + \tfrac{1}{4} \cdot log_2(\tfrac{1}{4})) & = 0{,}811 \\
I(raiz_{\text{civil=casado}}) = & \quad -(\tfrac{3}{6} \cdot log_2(\tfrac{3}{6}) + \tfrac{3}{6} \cdot log_2(\tfrac{3}{6})) & = 1 \\
I(raiz_{\text{civil}}) = & \quad \tfrac{4}{10} \cdot 0{,}811 + \tfrac{6}{10} \cdot 1 & = 0{,}925
\end{aligned}
$$

Índice de Gini

$$
\begin{aligned}
I(raiz_{\text{civil=solteiro}}) = & \quad 1 - ((\tfrac{3}{4})^2 + (\tfrac{1}{4})^2) & = 0{,}375 \\
I(raiz_{\text{civil=casado}}) = & \quad 1 - ((\tfrac{3}{6})^2 + (\tfrac{3}{6})^2) & = 0{,}500 \\
I(raiz_{\text{civil}}) = & \quad \tfrac{4}{10} \cdot 0{,}375 + \tfrac{6}{10} \cdot 0{,}5 & = 0{,}450
\end{aligned}
$$

`sexo`

Há 4 casos com `sexo` = `F`, 2 com empréstimo. Há 6 casos de homens, sendo que 2 deles conseguiram empréstimo:

$$
\begin{aligned}
I(raiz_{\text{sexo=F}}) = & \quad 1 - ((\tfrac{2}{4})^2 + (\tfrac{2}{4})^2)) & = 0{,}500 \\
I(raiz_{\text{sexo=M}}) = & \quad 1 - ((\tfrac{4}{6})^2 + (\tfrac{2}{6})^2)) & = 0{,}444 \\
I(raiz_{\text{sexo}}) = & \quad \tfrac{4}{10} \cdot 0{,}5 + \tfrac{6}{10} \cdot 0{,}444 & = 0{,}467
\end{aligned}
$$

```
round(impcol(2,2,4,2),3)
```

0.467

Nível Escolar (`educacao`)

O nível escolar (`educacao`) possui 3 categorias: `médio`, `superior` e `pós-graduação`. O cálculo é feito do mesmo modo:

Tabela 6: Valores para `educacao`

categoria	0	1	total
`médio`	3	0	3
`superior`	3	3	6
`pós-graduação`	0	1	1

$$
\begin{aligned}
I(raiz_{\text{educacao=médio}}) = & \quad 1 - ((\tfrac{3}{3})^2 + (\tfrac{0}{3})^2) & = 0 \\
I(raiz_{\text{educacao=superior}}) = & \quad 1 - ((\tfrac{3}{6})^2 + (\tfrac{3}{6})^2) & = 0{,}5 \\
I(raiz_{\text{educacao=pós-graduação}}) = & \quad 1 - ((\tfrac{0}{1})^2 + (\tfrac{1}{1})^2)) & = 0 \\
I(raiz_{\text{educacao}}) = & \quad \tfrac{3}{10} \cdot 0 + \tfrac{6}{10} \cdot 0{,}5 + \tfrac{1}{10} \cdot 0 & = 0{,}3
\end{aligned}
$$

Porém vamos trabalhar apenas com árvores binárias. A solução seria agrupar categorias de modo sempre ter duas saídas. Assim, as três categorias `médio`, `superior` e `pós-graduação` poderiam ser convertidas para três possibilidades binárias:

- `educacaoA` : `médio` $\times$ (`superior` | `pós-graduação`)
- `educacaoB` : `superior`$\times$ (`médio` | `pós-graduação`)
- `educacaoC` : `pós-graduação` $\times$ (`médio` | `superior`)

`educacaoA : médio` $\times$ (`superior` | `pós-graduação`)

Temos 3 casos de `médio` e nenhum conseguiu empréstimo; `superior` ou `pós-graduação` são 7 casos com 4 empréstimos:

$$
\begin{aligned}
I(raiz_{\text{educacaoA=médio}}) = & \quad 1 - ((\tfrac{3}{3})^2 + (\tfrac{0}{3})^2) & = 0 \\
I(raiz_{\text{educacaoA=superior|pós-graduação}}) = & \quad 1 - ((\tfrac{3}{7})^2 + (\tfrac{4}{7})^2) & = 0{,}490 \\
I(raiz_{\text{educacaoA}}) = & \quad \tfrac{3}{10} \cdot 0 + \tfrac{7}{10} \cdot 0{,}490 & = 0{,}343
\end{aligned}
$$

`educacaoB : superior` $\times$ (`médio` | `pós-graduação`)

Temos 6 casos de `superior` e 3 conseguiram empréstimo; `médio` ou `pós-graduação` são 4 casos com 1 empréstimo:

$$
\begin{aligned}
I(raiz_{\text{educacaoB=superior}}) = & \quad 1 - ((\tfrac{3}{6})^2 + (\tfrac{3}{6})^2) & = 0{,}500 \\
I(raiz_{\text{educacaoB=médio|pós-graduação}}) = & \quad 1 - ((\tfrac{3}{4})^2 + (\tfrac{1}{4})^2)) & = 0{,}375 \\
I(raiz_{\text{educacaoB}}) = & \quad \tfrac{6}{10} \cdot 0{,}5 + \tfrac{4}{10} \cdot 0{,}375 & = 0{,}450
\end{aligned}
$$

`educacaoC : pós-graduação` $\times$ (`médio` | `superior`)

Temos 1 caso de `pós-graduação` e obteve empréstimo; `médio` ou `superior` são 9 casos com 3 empréstimo:

$$
\begin{aligned}
I(raiz_{\text{educacaoC=pós-graduação}}) = & \quad 1 - ((\tfrac{0}{1})^2 + (\tfrac{1}{1})^2) & = 0 \\
I(raiz_{\text{educacaoC=médio|superior}}) = & \quad 1 - ((\tfrac{6}{9})^2 + (\tfrac{3}{9})^2)) & = 0{,}444 \\
I(raiz_{\text{educacaoC}}) = & \quad \tfrac{1}{10} \cdot 0 + \tfrac{9}{10} \cdot 0{,}444 & = 0{,}400
\end{aligned}
$$

Reparar que algumas bibliotecas para gerar árvores de decisão possuem um parâmetro que restringe o tamanho mínimo em uma separação, considerando um *desperdício* de nível uma decisão que separa o nó em x valores, $x < k$, sendo k um valor mínimo. Neste caso, a separação colocou de um lado 1 elemento e do outro 9.

credito

A variável `credito` (assim como `salario`) é contínua, não assumindo valores categóricos ou discretos. A técnica para construir a impureza dos filhos de um dado nó w é avaliar o melhor ponto de corte dentre os valores observados da variável.
A primeira ação é ordenar as observações pelo valor da variável:

```
print(exemplo[order(exemplo$credito),c(4,6)])
```

```
   credito emprest
9      5.6       1
4      6.3       0
6      6.9       0
7      7.2       0
8      7.6       0
3      7.8       1
10     7.9       0
5      8.5       0
2      9.3       1
1      9.5       1
```

Em seguida, é construída uma tabela com valores intermediários (e externos, no caso do valor inferior e superior). O primeiro ponto de corte seria antes do primeiro valor de `credito`: 5,6. O valor é arbitrário, qualquer valor menor que 5,6 serve. Foi utilizada a mesma distância entre o primeiro e o segundo valor $(5{,}6-(6{,}3-5{,}6)/2 = 5{,}25)$ para o primeiro e a mesma lógica para o último.
Agora é calculada o valor da impureza - entropia, no nosso caso - para cada limite de corte, inclusive o inferior e superior.

Tabela 7: Separação em classes para cálculo da Impureza coletiva por cada corte em `credito`

emprest		**1**		0		**0**		0		**0**		1		**0**		0		**1**		1			
credito		**5,6**		6,3		**6,9**		7,2		**7,6**		7,8		**7,9**		8,5		**9,3**		9,5			
		$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$
lim.		**5,2**	**5,2**	5,9	5,9	**6,6**	**6,6**	7,1	7,1	**7,4**	**7,4**	7,7	7,7	**7,85**	**7,85**	8,2	8,2	**8,9**	**8,9**	9,4	9,4	**9,6**	**9,6**
	0	**0**	**6**	0	6	**1**	**5**	2	4	**3**	**3**	4	2	**4**	**2**	5	1	**6**	**0**	6	0	**6**	**0**
	1	**0**	**4**	1	3	**1**	**3**	1	3	**1**	**3**	1	3	**2**	**2**	2	2	**2**	**2**	3	1	**4**	**0**

$$I(raiz_{\text{credito}\leq 5{,}2}) = \quad \text{como há uma divisão por 0, convencionou-se ser 0} \quad = 0$$

$$I(raiz_{\text{credito}>5{,}2}) = \quad 1-((\tfrac{6}{10})^2+(\tfrac{4}{10})^2)) \quad = 0{,}480$$

$$I(raiz_{\text{credito_5,2}}) = \quad \tfrac{0}{10}\cdot 0+\tfrac{10}{10}\cdot 0{,}48 \quad = 0{,}480$$

Observação: a primeira e a última supõe a criação de um nó com 0 observações. A convenção da entropia para esses casos é 0. Na prática, os algoritmos de construção de árvores de decisão alteram os valores para que nunca ocorra essa situação.

$$I(raiz_{\text{credito}\leq 5{,}9}) = \quad 1-((\tfrac{0}{1})^2+(\tfrac{1}{1})^2) \quad = 0$$

$$I(raiz_{\text{credito}>5{,}9}) = \quad 1-((\tfrac{6}{9})^2+(\tfrac{3}{9})^2)) \quad = 0{,}444$$

$$I(raiz_{\text{credito_5,9}}) = \quad \tfrac{1}{10}\cdot 0+\tfrac{9}{10}\cdot 0{,}444 \quad = 0{,}400$$

$$
\begin{aligned}
I(raiz_{\texttt{credito}\leq 6{,}6}) = &\quad 1-((\tfrac{1}{2})^2+(\tfrac{1}{2})^2) &&= 0{,}5\\
I(raiz_{\texttt{credito}>6{,}6}) = &\quad 1-((\tfrac{5}{8})^2+(\tfrac{3}{8})^2)) &&= 0{,}469\\
I(raiz_{\texttt{credito_}6{,}6}) = &\quad \tfrac{2}{10}\cdot 0{,}5+\tfrac{8}{10}\cdot 0{,}469 &&= 0{,}475
\end{aligned}
$$

$$
\begin{aligned}
I(raiz_{\texttt{credito}\leq 7{,}1}) = &\quad 1-((\tfrac{2}{3})^2+(\tfrac{1}{3})^2)) &&= 0{,}444\\
I(raiz_{\texttt{credito}>7{,}1}) = &\quad 1-((\tfrac{4}{7})^2+(\tfrac{3}{7})^2) &&= 0{,}490\\
I(raiz_{\texttt{credito_}7{,}1}) = &\quad \tfrac{3}{10}\cdot 0{,}444+\tfrac{7}{10}\cdot 0{,}490 &&= 0{,}476
\end{aligned}
$$

...

$$
\begin{aligned}
I(raiz_{\texttt{credito}\leq 9{,}6}) = &\quad 1-((\tfrac{6}{10})^2+(\tfrac{4}{10})^2)) &&= 0{,}480\\
I(raiz_{\texttt{credito}>9{,}6}) = &\quad \text{como há uma divisão por 0, convencionou-se ser 0} &&= 0\\
I(raiz_{\texttt{credito_}9{,}6}) = &\quad \tfrac{10}{10}\cdot 0{,}480+\tfrac{0}{10}\cdot 0 &&= 0{,}480
\end{aligned}
$$

É possível então construir uma tabela com a impureza coletiva (ic) dos filhos para cada corte em `credito`:

Tabela 8: Impureza coletiva em cortes de `credito`

cortes	5,2	5,9	6,6	7,1	7,4	7,7	7,85	8,2	**8,9**	9,4	9,6
ic	0,480	0,400	0,475	0,476	0,450	0,400	0,467	0,419	**0,300**	0,400	0,480

Podemos observar que o corte que produz a menor impureza coletiva dos filhos do nó raiz é `credito` $\leq 8{,}9$ com impureza coletiva de 0,300.

$$I(raiz_{\texttt{credito}}) = 0{,}300$$

salario

A variável `salario` é contínua e o procedimento é idêntico àquele aplicado ao `credito`.
A primeira ação é ordenar as observações pelo valor da variável:

```
print(exemplo[order(exemplo$salario),c(5,6)])
```

```
    salario emprest
7   7415.93       0
10  7638.89       0
8   8446.85       0
5   9561.46       0
2  10546.73       1
9  11234.57       1
4  12428.51       0
6  16623.75       0
1  17479.44       1
3  29345.74       1
```

É possível então construir uma tabela com a impureza coletiva (ic) dos filhos para cada corte em `salario`:

Tabela 9: Cortes `salario`

cortes	7.300	7.500	8.000	9.000	**10.000**	11.000	12.000	14.000	17.000	23.000	31.000
ic	0,480	0,444	0,400	0,343	**0,267**	0,400	0,467	0,419	0,300	0,400	0,480

Podemos observar que o corte que produz a menor impureza coletiva dos filhos do nó raiz é `salario` ≤ 10000 com impureza coletiva de 0,267.

$$I(raiz_{\text{salario}}) = 0{,}267$$

Escolha da decisão

Vamos analisar as impurezas coletivas e seus valores:

Tabela 10: Compilação das Impurezas coletivas das variáveis

variável	$I(raiz_{\text{variavel}})$
$raiz_{\text{civil}}$	0,450
$raiz_{\text{sexo}}$	0,467
$raiz_{\text{educacaoA}}$	0,343
$raiz_{\text{credito}<8{,}9}$	0,300
$raiz_{\text{salario}<10.000}$	0,267

A impureza da raiz era:

$$I(raiz) = 1 - ((\frac{6}{10})^2 + (\frac{4}{10})^2)) = 0{,}480$$

O melhor teste é aquele que maximiza a redução da impureza do nó-pai (o raiz, neste exemplo).
Podemos chamar de Δ a diferença entre a impureza original (pai) e a atual (filhos) e é, portanto, um ganho em pureza pela divisão.

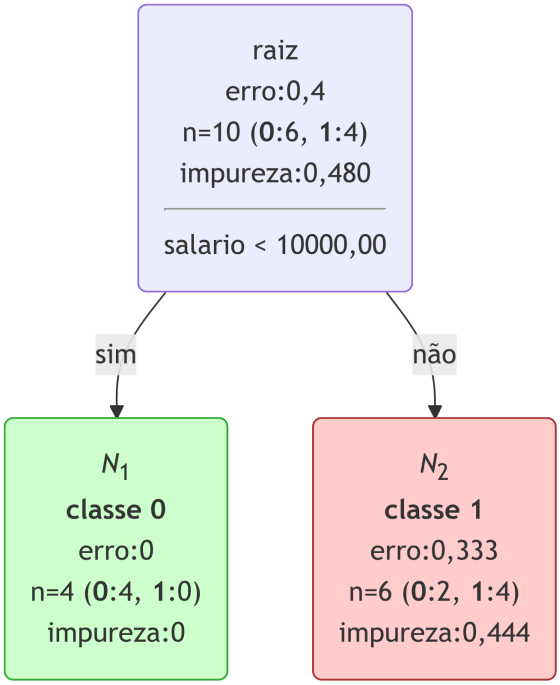
$$\Delta = I(pai) - I(filhos)$$

Para cada uma das variáveis podemos calcular a diferença:

Tabela 11: Diferença (ganho) entre a impureza do pai e dos filhos

variável	$I(raiz_{\text{variavel}})$	Δ
$raiz_{\text{civil}}$	0,450	0,480 - 0,450 = 0,030
$raiz_{\text{sexo}}$	0,467	0,013
$raiz_{\text{educacaoA}}$	0,343	0,137
$raiz_{\text{credito}<8,9}$	0,300	0,180
$raiz_{\text{salario}<10.000}$	0,267	0,213

A escolha então que aumenta a pureza daquele nó (raiz) é `salario` < 10.000



A impureza de N_1 é 0 e, portanto, não há mais o que fazer. A impureza de N_2 é 0,444 e está passível de todo o processo ser refeito agora considerando N_2 o nó-pai e procurando a melhor divisão para os seus nós-filhos.

```
a0 <- 4; a1 <- 2
b0 <- 0; b1 <- 1
round(gini(a0,a1),3)
round(gini(b0,b1),3)
round(impcol(a0,a1,b0,b1),3)
```

0.444

0

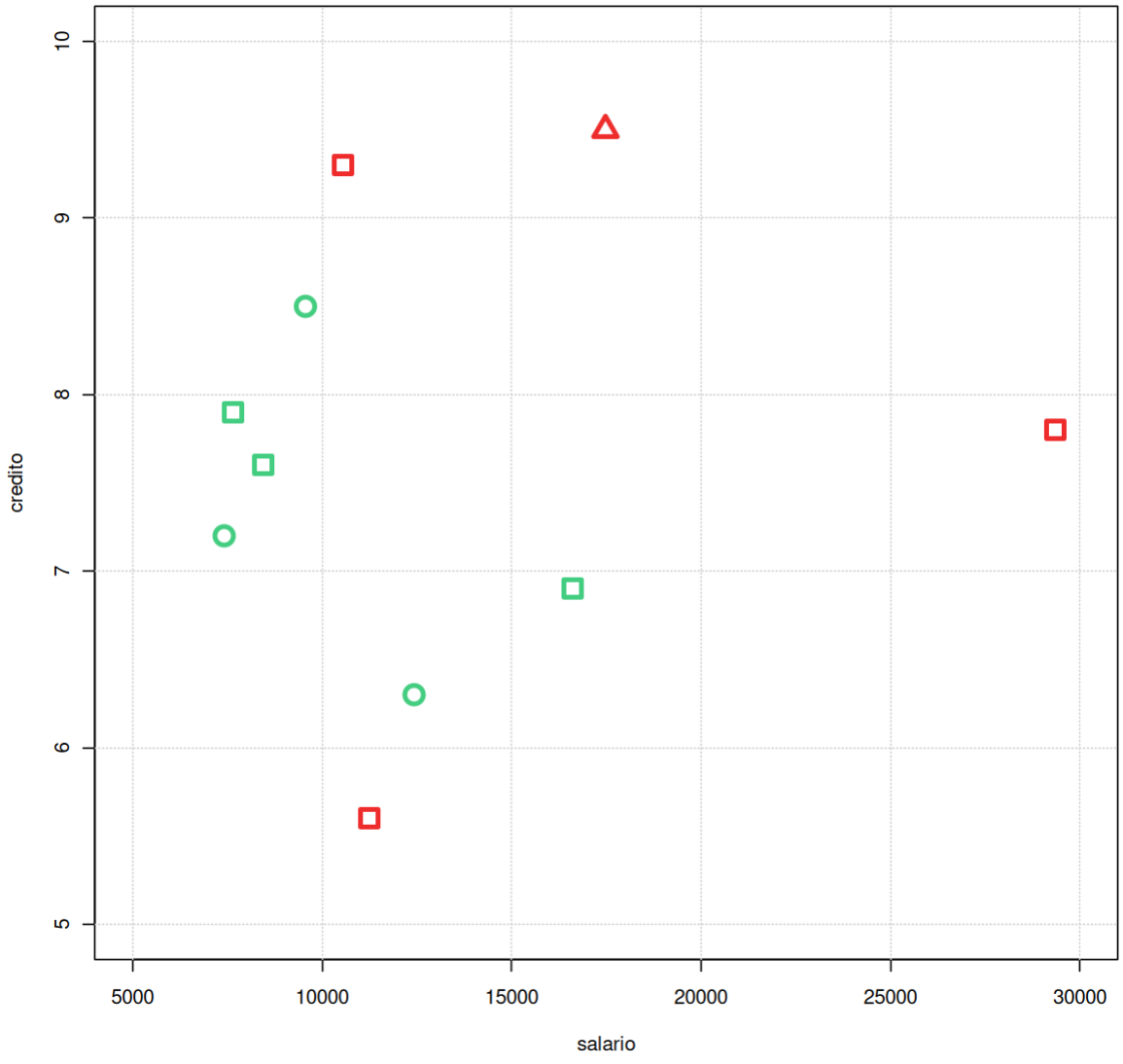
0.381

Vamos analisar as impurezas coletivas e seus valores:

Tabela 12: Compilação das Impurezas coletivas das variáveis

variável	$I(raiz_{\text{variavel}})$
$raiz_{\text{civil}}$	0,924
$raiz_{\text{sexo}}$	0,951
$raiz_{\text{educacaoA}}$	0,690
$raiz_{\text{credito}}$	0,649
$raiz_{\text{salario}}$	0,551

```
grafdim(10,10)
cor_classe <- c("seagreen3",        # 0 - negativo, sem empréstimo
                "firebrick2")   # 1 - positivo, emprétimo concedido
tipo_ponto <- c(1,2,0) # o → médio, Δ → superior,  □ → pós
plot(empr$salario,empr$credito, cex=2, lwd=3,
     xlim=c(5000,30000), ylim=c(5,10), xlab="salario", ylab="credito",
     col=cor_classe[as.numeric(empr$emprest)],
     pch=tipo_ponto[as.numeric(empr$educacao)]) # pode ser empr$sexo ou civil
grid(nx=NULL)
```



Utilizando a biblioteca `rpart` que implementa em R a construção recursiva de árvores de decisão:

```r
modelo <- rpart(emprest ~ civil + sexo + educacao + credito + salario, # modelo
                data=exemplo,                                          # dados
                method="class",                                        # classificação
                control=rpart.control(minsplit=2, minbucket=0))        # parâmetros de controle
```

Algund parâmetros da criação da árvore de `rpart` também permitem fazer uma *pré-poda* na árvore:

- `minsplit`: especifica o mínimo de observações em um nó para que haja uma uma divisão. Por exemplo, se `minsplit` $= 5$, um nó com 5 ou menos observações automaticamente está finalizado;
- `maxdepth`: especifica a profundidade máxima da árvore. A árvore pára de crescer ao atingir `maxdepth`;
- `minbucket`: define a quantidade mínima de observações em uma folha. Se for tentada uma divisão e aparecer uma folha com menos que `minbucket`, a decisão é cancelada.

```r
print(modelo)
```

```
n= 10

node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 10 4 0 (0.6000000 0.4000000)
   2) salario< 10054.09 4 0 0 (1.0000000 0.0000000) *
   3) salario≥10054.09 6 2 1 (0.3333333 0.6666667)
     6) credito< 7.35 3 1 0 (0.6666667 0.3333333)
      12) credito≥5.95 2 0 0 (1.0000000 0.0000000) *
      13) credito< 5.95 1 0 1 (0.0000000 1.0000000) *
     7) credito≥7.35 3 0 1 (0.0000000 1.0000000) *
```

É possível ver até onde chegamos com os cálculos manuais: quando o nó raiz se dividiu nos nós 2 e 3 em função do `salario` < 10000. O nó 2 tem um asterisco `*` no final, indicando ser uma folha com probabilidades da classe `0` ser 1, e da classe `1`, ser 0.

Vamos analisar o resultado dos passos do processo:

```r
printcp(modelo)
```

```
Classification tree:
rpart(formula = emprest ~ civil + sexo + educacao + credito +
    salario, data = exemplo, method = "class", control = rpart.control(minsplit = 2,
    minbucket = 0))

Variables actually used in tree construction:
[1] credito salario

Root node error: 4/10 = 0.4

n= 10

    CP nsplit rel error xerror    xstd
1 0.50      0       1.0   1.00 0.38730
2 0.25      1       0.5   2.25 0.23717
3 0.01      3       0.0   2.00 0.31623
```

O erro da raiz é 0,4, pois considerou todas observações da classe 0 e há 4 da classe 1.
Se for desejado estudar cada nó e sua divisão, podemos pedir o resumo do modelo:

```
summary(modelo)
```

```
Call:
rpart(formula = emprest ~ civil + sexo + educacao + credito +
    salario, data = exemplo, method = "class", control = rpart.control(minsplit = 2,
    minbucket = 0))
  n= 10

    CP nsplit rel error xerror      xstd
1 0.50      0       1.0   1.00 0.3872983
2 0.25      1       0.5   2.25 0.2371708
3 0.01      3       0.0   2.00 0.3162278

Variable importance
 credito  salario educacao     sexo
      44       42        7        6

Node number 1: 10 observations,    complexity param=0.5
  predicted class=0  expected loss=0.4  P(node) =1
    class counts:     6     4
   probabilities: 0.600 0.400
  left son=2 (4 obs) right son=3 (6 obs)
  Primary splits:
      salario  < 10054.09 to the left,  improve=2.1333330, (0 missing)
      credito  < 8.9      to the left,  improve=1.8000000, (0 missing)
      educacao splits as  LRR,          improve=1.3714290, (0 missing)
      civil    splits as  RL,           improve=0.3000000, (0 missing)
      sexo     splits as  RL,           improve=0.1333333, (0 missing)
  Surrogate splits:
      educacao splits as  LRR,          agree=0.7, adj=0.25, (0 split)
      credito  < 7.05     to the right, agree=0.7, adj=0.25, (0 split)

Node number 2: 4 observations
  predicted class=0  expected loss=0  P(node) =0.4
    class counts:     4     0
   probabilities: 1.000 0.000

Node number 3: 6 observations,    complexity param=0.25
  predicted class=1  expected loss=0.3333333  P(node) =0.6
    class counts:     2     4
   probabilities: 0.333 0.667
  left son=6 (3 obs) right son=7 (3 obs)
  Primary splits:
      credito  < 7.35     to the left,  improve=1.3333330, (0 missing)
      educacao splits as  LRR,          improve=1.0666670, (0 missing)
      salario  < 11831.54 to the right, improve=0.6666667, (0 missing)
      civil    splits as  RL,           improve=0.1666667, (0 missing)
  Surrogate splits:
      salario < 17051.6  to the left,  agree=0.833, adj=0.667, (0 split)
      sexo    splits as  RL,           agree=0.667, adj=0.333, (0 split)

Node number 6: 3 observations,    complexity param=0.25
  predicted class=0  expected loss=0.3333333  P(node) =0.3
```

```
    class counts:     2     1
   probabilities: 0.667 0.333
  left son=12 (2 obs) right son=13 (1 obs)
  Primary splits:
      credito  < 5.95     to the right, improve=1.3333330, (0 missing)
      salario  < 11831.54 to the right, improve=1.3333330, (0 missing)
      civil    splits as  RL,           improve=0.3333333, (0 missing)
      sexo     splits as  LR,           improve=0.3333333, (0 missing)
      educacao splits as  LLR,          improve=0.3333333, (0 missing)

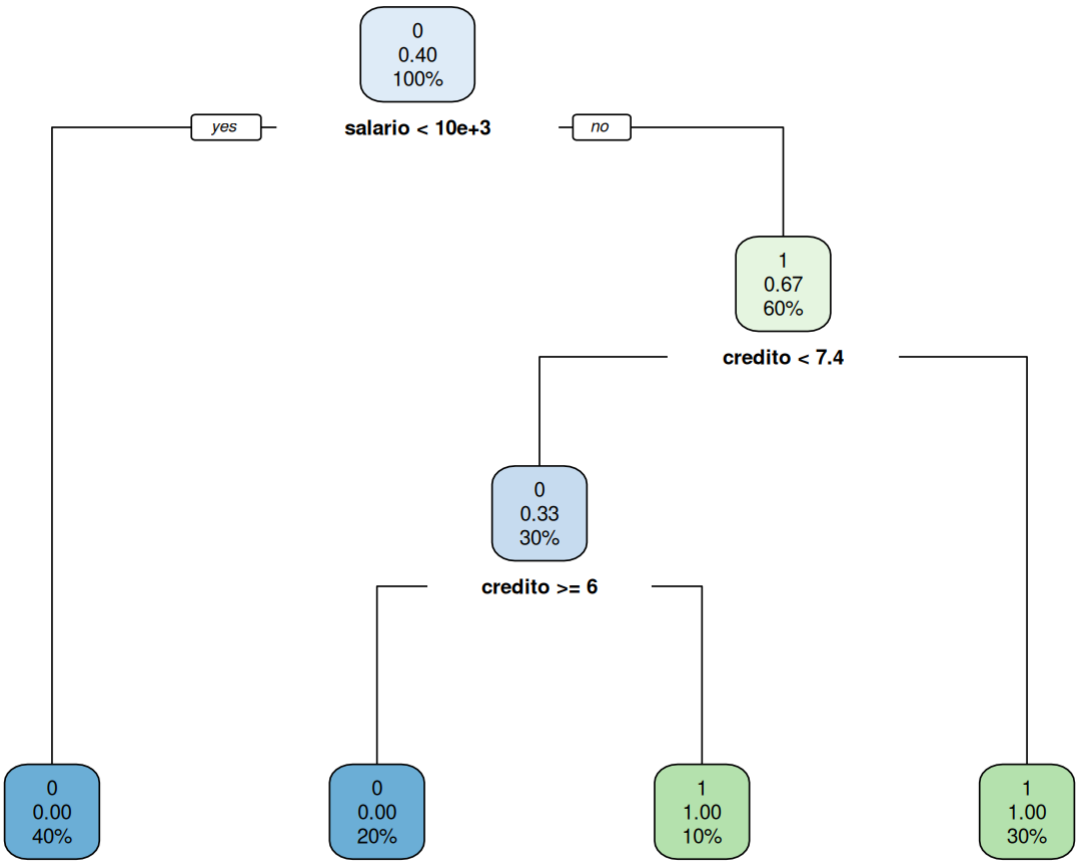
Node number 7: 3 observations
  predicted class=1  expected loss=0  P(node) =0.3
    class counts:     0     3
   probabilities: 0.000 1.000

Node number 12: 2 observations
  predicted class=0  expected loss=0  P(node) =0.2
    class counts:     2     0
   probabilities: 1.000 0.000

Node number 13: 1 observations
  predicted class=1  expected loss=0  P(node) =0.1
    class counts:     0     1
   probabilities: 0.000 1.000
```

Por fim, é possível pedir a representação gráfica da árvore:

```
grafdim(10,8)
rpart.plot(modelo)
```



Dados de treino, teste e validação

Construção dos conjuntos de treino, teste e validação

```
empr <- criar_emprestimo(1000)
```

```
(num_obs   <- nrow(empr))
tx_teste   <- 0.17
tx_valid   <- 0.17
(num_teste <- trunc(num_obs * tx_teste))
(num_valid <- trunc(num_obs * tx_valid))
(num_trein <- num_obs - num_teste - num_valid)
```

1000

170

170

660

```
treino <- empr
amostra  <- sample(1:num_obs, num_teste, replace=FALSE)
teste  <- treino[ amostra, ]
treino <- treino[-amostra, ]
amostra  <- sample(1:(num_obs - num_teste), num_valid, replace=FALSE)
valid  <- treino[ amostra, ]
treino <- treino[-amostra, ]
head(treino)
head(valid)
```

A data.frame: 6 × 6

	civil <fct>	sexo <fct>	educacao <ord>	credito <dbl>	salario <dbl>	emprest <fct>
1	casado	M	superior	9.1	10502.86	1
2	casado	M	superior	6.9	9064.96	0
3	solteiro	M	médio	8.3	10346.63	0
4	casado	F	médio	5.8	9476.84	0
5	solteiro	M	médio	6.3	5761.68	0
7	casado	F	superior	6.6	9255.89	0

A data.frame: 6 × 6

	civil <fct>	sexo <fct>	educacao <ord>	credito <dbl>	salario <dbl>	emprest <fct>
835	casado	F	superior	8.7	10099.59	1
177	casado	M	superior	7.3	14927.83	1
994	casado	M	médio	8.8	13959.40	0
361	casado	M	superior	6.2	10022.90	1
890	solteiro	M	pós-graduação	9.1	12080.10	1
678	solteiro	M	superior	8.3	15211.66	0

Modelo de Árvore de Decisão

Aderindo um modelo através de `rpart`:

```
modelo = rpart(emprest ~ ., data=treino, control=rpart.control(cp = 0.0001))
```

```
printcp(modelo)
```

```
Classification tree:
rpart(formula = emprest ~ ., data = treino, control = rpart.control(cp = 1e-04))

Variables actually used in tree construction:
[1] civil    credito  educacao salario

Root node error: 259/660 = 0.39242

n= 660

        CP nsplit rel error  xerror     xstd
1 0.285714      0  1.000000 1.00000 0.048434
2 0.254826      1  0.714286 0.89575 0.047358
3 0.115830      2  0.459459 0.48263 0.038865
4 0.030888      3  0.343629 0.35135 0.034198
5 0.027027      8  0.189189 0.25483 0.029757
6 0.003861     11  0.104247 0.17761 0.025258
7 0.000100     13  0.096525 0.18919 0.026004
```

```
modelo
```

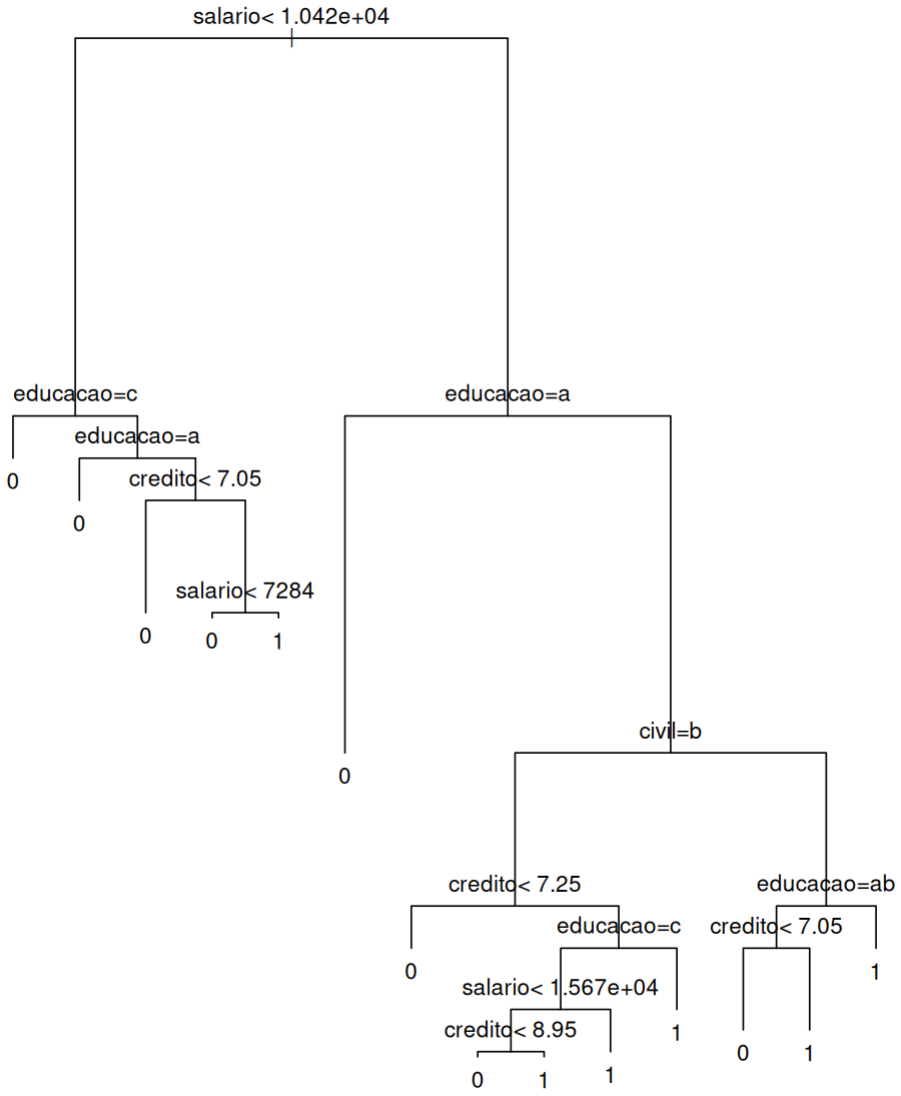
```
n= 660

node), split, n, loss, yval, (yprob)
      * denotes terminal node

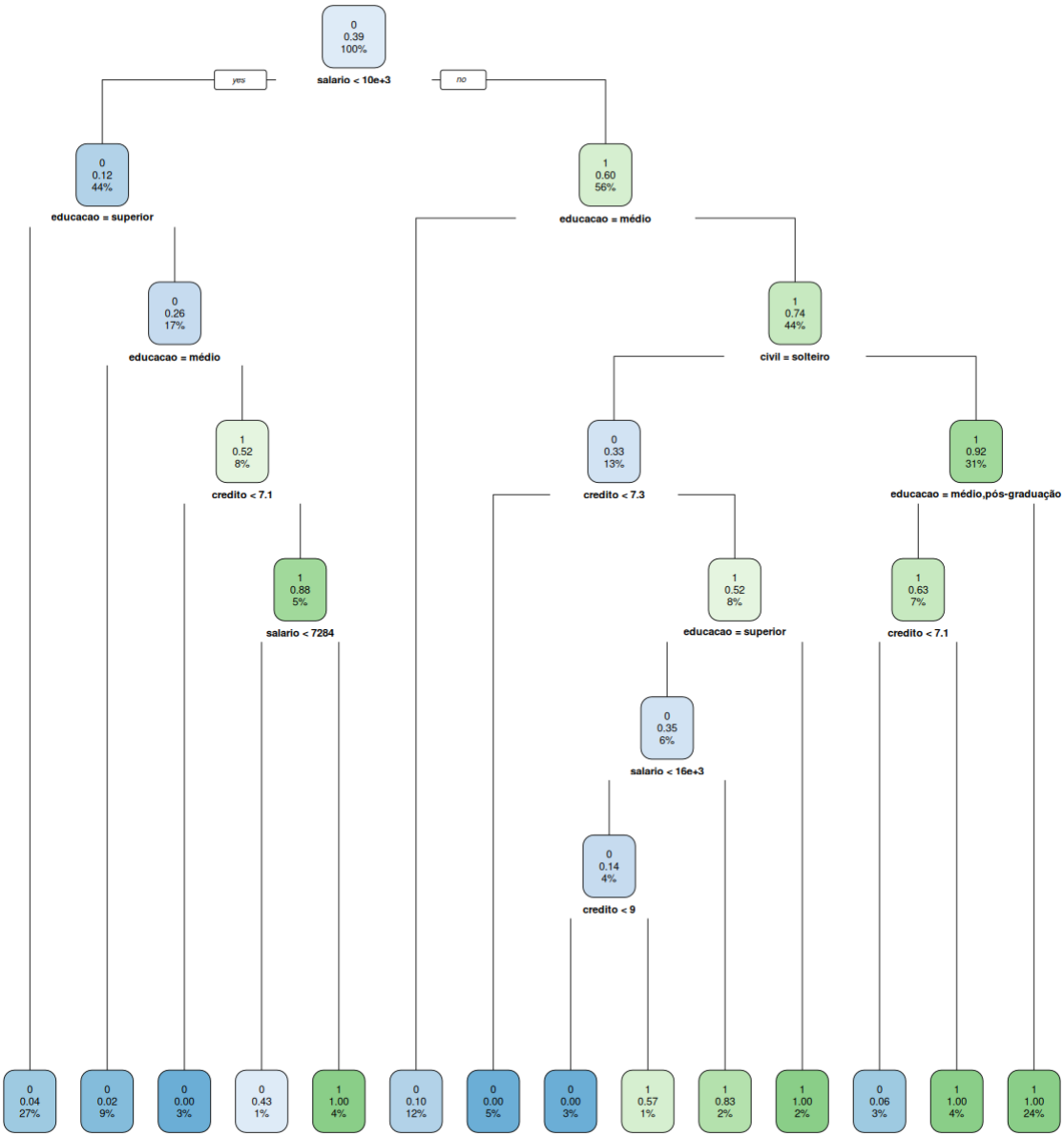
  1) root 660 259 0 (0.60757576 0.39242424)
    2) salario< 10415.8 288  36 0 (0.87500000 0.12500000)
      4) educacao=superior 177   7 0 (0.96045198 0.03954802) *
      5) educacao=médio,pós-graduação 111  29 0 (0.73873874 0.26126126)
       10) educacao=médio 57   1 0 (0.98245614 0.01754386) *
       11) educacao=pós-graduação,superior 54  26 1 (0.48148148 0.51851852)
         22) credito< 7.05 22   0 0 (1.00000000 0.00000000) *
         23) credito≥7.05 32   4 1 (0.12500000 0.87500000)
           46) salario< 7283.835 7   3 0 (0.57142857 0.42857143) *
           47) salario≥7283.835 25   0 1 (0.00000000 1.00000000) *
    3) salario≥10415.8 372 149 1 (0.40053763 0.59946237)
      6) educacao=médio 82   8 0 (0.90243902 0.09756098) *
      7) educacao=pós-graduação,superior 290  75 1 (0.25862069 0.74137931)
       14) civil=solteiro 86  28 0 (0.67441860 0.32558140)
         28) credito< 7.25 32   0 0 (1.00000000 0.00000000) *
         29) credito≥7.25 54  26 1 (0.48148148 0.51851852)
           58) educacao=superior 40  14 0 (0.65000000 0.35000000)
            116) salario< 15669.66 28   4 0 (0.85714286 0.14285714)
              232) credito< 8.95 21   0 0 (1.00000000 0.00000000) *
              233) credito≥8.95 7   3 1 (0.42857143 0.57142857) *
            117) salario≥15669.66 12   2 1 (0.16666667 0.83333333) *
```

```
      59) educacao=médio,pós-graduação 14   0 1 (0.00000000 1.00000000) *
    15) civil=casado 204  17 1 (0.08333333 0.91666667)
      30) educacao=médio,pós-graduação 46  17 1 (0.36956522 0.63043478)
        60) credito< 7.05 18   1 0 (0.94444444 0.05555556) *
        61) credito≥7.05 28   0 1 (0.00000000 1.00000000) *
      31) educacao=superior 158   0 1 (0.00000000 1.00000000) *
```

```r
grafdim(9,11)
plot(modelo)
text(modelo, cex=1.1, xpd=TRUE)
```




```r
grafdim(11,11)
rpart.plot(modelo)
```




```
predito <- predict(modelo, valid, type="class")
```

```
head(data.frame(valid$emprest, predito),10)
```

A data.frame: 10 × 2

	valid.emprest <fct>	predito <fct>
835	1	0
177	1	1
994	0	0
361	1	0
890	1	1
678	0	0
990	0	0
530	0	0
88	1	1
267	0	0

Qualidade do modelo

```
confusionMatrix(reference = valid$emprest, data=predito, positive="1")
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 101   9
         1   2  58

               Accuracy : 0.9353
                 95% CI : (0.8872, 0.9673)
    No Information Rate : 0.6059
    P-Value [Acc > NIR] : < 2e-16

                  Kappa : 0.862

 Mcnemar's Test P-Value : 0.07044

            Sensitivity : 0.8657
            Specificity : 0.9806
         Pos Pred Value : 0.9667
         Neg Pred Value : 0.9182
             Prevalence : 0.3941
         Detection Rate : 0.3412
   Detection Prevalence : 0.3529
      Balanced Accuracy : 0.9231

       'Positive' Class : 1
```

Modelo de Árvore de Decisão Podada

```
modelo.poda = prune(modelo, cp=0.22)
printcp(modelo.poda)
```

```
Classification tree:
rpart(formula = emprest ~ ., data = treino, control = rpart.control(cp = 1e-04))

Variables actually used in tree construction:
[1] educacao salario

Root node error: 259/660 = 0.39242

n= 660

       CP nsplit rel error  xerror     xstd
1 0.28571      0   1.00000 1.00000 0.048434
2 0.25483      1   0.71429 0.89575 0.047358
3 0.22000      2   0.45946 0.48263 0.038865
```

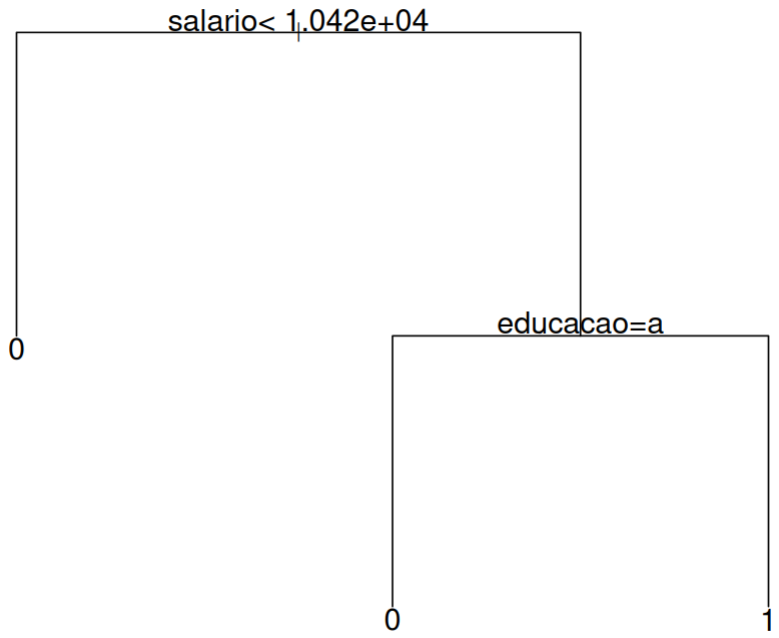
```
modelo.poda
```

```
n= 660

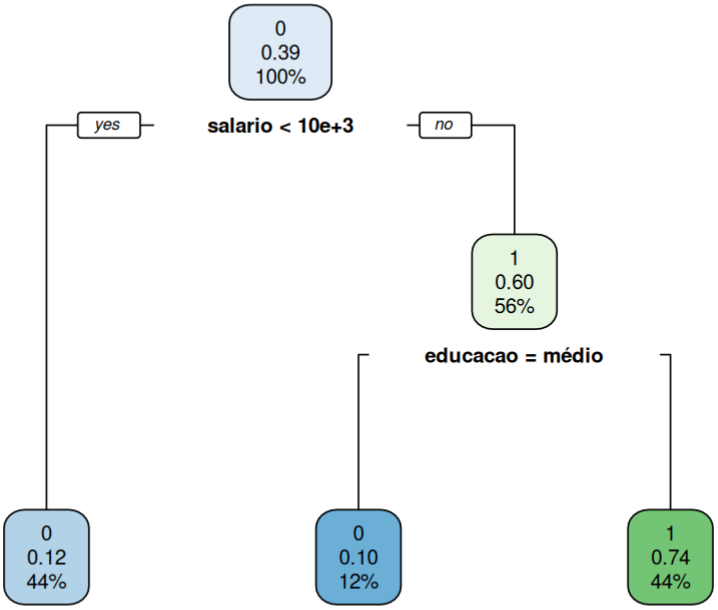
node), split, n, loss, yval, (yprob)
      * denotes terminal node

1) root 660 259 0 (0.60757576 0.39242424)
  2) salario< 10415.8 288  36 0 (0.87500000 0.12500000) *
  3) salario≥10415.8 372 149 1 (0.40053763 0.59946237)
    6) educacao=médio 82   8 0 (0.90243902 0.09756098) *
    7) educacao=pós-graduação,superior 290  75 1 (0.25862069 0.74137931) *
```

```
grafdim(8,7)
plot(modelo.poda)
text(modelo.poda, cex=1.5, xpd=TRUE)
```




```
grafdim(9,8)
rpart.plot(modelo.poda)
```



Verificar qualidade do modelo com os dados de validação:

```
predito <- predict(modelo.poda, valid, type="class")
```

```
head(data.frame(valid$emprest, predito),10)
```

A data.frame: 10 × 2

	valid.emprest <fct>	predito <fct>
835	1	0
177	1	1
994	0	0
361	1	0
890	1	1
678	0	1
990	0	1
530	0	0
88	1	1
267	0	0

Qualidade do modelo

```
confusionMatrix(reference = valid$emprest, data=predito)
```

```
Confusion Matrix and Statistics

          Reference
```

```
Prediction  0  1
         0 79 14
         1 24 53

               Accuracy : 0.7765
                 95% CI : (0.7063, 0.8367)
    No Information Rate : 0.6059
    P-Value [Acc > NIR] : 1.762e-06

                  Kappa : 0.5438

 Mcnemar's Test P-Value : 0.1443

            Sensitivity : 0.7670
            Specificity : 0.7910
         Pos Pred Value : 0.8495
         Neg Pred Value : 0.6883
             Prevalence : 0.6059
         Detection Rate : 0.4647
   Detection Prevalence : 0.5471
      Balanced Accuracy : 0.7790

       'Positive' Class : 0
```

Conjunto de modelos com `caret::train`

```
controles <- trainControl(method="cv", number=10)
```

```
modelo.caret <- caret::train(emprest ~ ., data=treino, trControl=controles, method="rpart")
```

```
modelo.caret
```

```
CART

660 samples
  5 predictor
  2 classes: '0', '1'

No pre-processing
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 593, 595, 594, 594, 594, 594, ...
Resampling results across tuning parameters:

  cp         Accuracy   Kappa
  0.1158301  0.8227895  0.6266350
  0.2548263  0.7365602  0.4843137
  0.2857143  0.6498469  0.2016005

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was cp = 0.1158301.
```

```
predito <- predict(modelo.caret, valid)
```

```
confusionMatrix(reference=valid$emprest, data=predito)
```

```
Confusion Matrix and Statistics

          Reference
Prediction  0  1
         0 79 14
         1 24 53

               Accuracy : 0.7765
                 95% CI : (0.7063, 0.8367)
    No Information Rate : 0.6059
    P-Value [Acc > NIR] : 1.762e-06

                  Kappa : 0.5438

 Mcnemar's Test P-Value : 0.1443

            Sensitivity : 0.7670
            Specificity : 0.7910
         Pos Pred Value : 0.8495
         Neg Pred Value : 0.6883
             Prevalence : 0.6059
         Detection Rate : 0.4647
   Detection Prevalence : 0.5471
      Balanced Accuracy : 0.7790

       'Positive' Class : 0
```

Caso 2 - Fraude no Seguro

Referência

Bibliotecas

```
rm(list=ls())
library(rpart)  # árvores de decisão
library(repr)   # representações (ajuste gráfico do jupyter)
library(caret)  # matriz de confusão

library(rpart.plot)
library(RColorBrewer)
```

```
grafdim <- function(x=11, y=9) { options(repr.plot.width=x, repr.plot.height=y) }
grafdim()
```

Preparação do ambiente

```
set.seed(1701)
```

Dados

```r
tamanho <- 1000
empr <- data.frame(
  # 30% de solteiros, 70% de casados
  civil    = factor(sample(c('solteiro','casado'), tamanho,
                           replace=TRUE, prob=c(0.3, 0.7))),
  # 50% de sexo 'M', 50% de 'F'
  sexo     = factor(sample(c('M','F'), tamanho,
                           replace=TRUE, prob=c(0.5, 0.5))),
  # 20% com ensino médio, 60% com superior e 20% com pós
  educacao = factor(sample(c("médio", "superior", "pós-graduação"),
                           tamanho, replace=TRUE,
                           prob=c(0.2, 0.6, 0.2)), ordered=TRUE),
  # crédito tem distribuição uniforme [0,55; 0,95]
  credito  = round(runif(tamanho, 0.55, 0.95), 1) * 10,
  # salário tem distribuição Weibull com forma 1,6 e escala 2
  # para ser assimétrico, com cauda longa à direita
  salario  = round(rweibull(tamanho,1.6,2)*40,3)*100 + 5000
)
# "regras" (secretas) para conceder um empréstimo
empr$emprest <- factor(as.numeric(
    empr$civil == 'solteiro' & (empr$sexo == 'F' & empr$credito > 7.5 & empr$salario > 15000 |
                                empr$sexo == 'M' & empr$credito > 9.0) |
    empr$salario > 25000 & empr$credito > 0.7 |
    empr$educacao == 'superior' & empr$salario > 10000 &
                    empr$credito > 0.6 & empr$civil == 'casado' |
    empr$educacao == 'pós-graduação' & empr$credito > 7.0 & empr$salario > 7000
    ))
```

```r
summary(empr)
```

```r
head(empr, 10)
```

```r
modelo = rpart(emprest ~ civil + sexo + educacao + credito + salario, data=empr)
```

```r
modelo
```

```r
printcp(modelo)
```

```r
grafdim(12,10)
rpart.plot(modelo)
```

```r
treino <- data.frame(
  ClaimID = c(1,2,3),
  RearEnd = c(TRUE, FALSE, TRUE),
  Fraud   = c(TRUE, FALSE, TRUE) )
```

```r
treino
```

Modelo

```r
tree <- rpart(Fraud ~ RearEnd, data=treino, method="class")
```

```
printcp(tree)
```

```
tree
```

A análise criou apenas o nó raiz pois os parâmetros por omissão de `rpart` preveniram a árvore de crescer, especificamente `minsplit` e `minbucket`.

```
tree <- rpart(Fraud ~ RearEnd, data=treino,
              method="class",
              minsplit=2, minbucket=1)
```

```
printcp(tree)
```

```
grafdim(8,7)
plot(tree)
text(tree, cex=1.3, xpd=TRUE)
```

```
grafdim(9,8)
rpart.plot(tree, cex=1.3)
```

`rpart` utiliza índice de Gini como indicador de impureza para dividir os nós. Pode ser usado o ganho de informação (entropia):

```
tree.info <- rpart(Fraud ~ RearEnd, data=treino,
                   method="class",
                   parms=list(split = 'information'),
                   minsplit=2, minbucket=1)
```

```
printcp(tree.info)
```

```
tree.info
```

Com mais impureza:

Modelo com peso

```
treino <- data.frame(
  ClaimID = c(   1,     2,     3,     4,    5,     6,     7),
  RearEnd = c(TRUE, FALSE,  TRUE, FALSE, TRUE,  TRUE, FALSE),
  Fraud   = c(TRUE, FALSE, FALSE,  TRUE, TRUE, FALSE, FALSE))
```

```
treino
```

```
tree <- rpart(Fraud ~ RearEnd, data=treino,
              method="class",
              minsplit=2, minbucket=1)
```

```
printcp(tree)
```

O algoritmo de crescimento da árvore do `rpart` não crescerá a árvore se sua complexidade crescer mais que a capacidade de explicação do modelo, ou seja, se o tamanho da árvore aumentar mas não sua capacidade de separação de classes. Para impedir que o algoritmo interrompa o crescimento (ou seja, garanta o crescimento) é necessário dar um valor negativo para o parâmetro `cp`.

```
tree <- rpart(Fraud ~ RearEnd, data=treino,
              method="class",
              minsplit=2, minbucket=1, cp=-1)
```

```
printcp(tree)
```

```
grafdim(9,8)
rpart.plot(tree, cex=1.3)
```

Fica claro que crescer a árvore não proporcionou ganho. Havia 43% (3/7) de casos de fraude. A árvore recomenda: se não for batida traseira (`RearEnd = 0`, ou seja `FALSE`) considere não fraude (com erro de 0,33) e se for batida traseira (`RearEnd ≠ 0`, ou seja `TRUE`), considere não fraude com erro de 50%.

Forçar o crescimento da árvore em geral produz modelo com *overfitting* .

Pode-se fornecer um peso para cada observação, indicando a importância dela para a modelagem:

```
tree <- rpart(Fraud ~ RearEnd, data=treino, method="class",
              minsplit=2, minbucket=1,
              weights=c(0.16,0.16,0.12,0.12,0.16,0.12,0.16), cp=-1)
```

```
printcp(tree)
```

```
rpart.plot(tree, cex=1.3)
```

Se não for batida traseira, considere que não houve fraude, com erro de 23%; se foi batida traseira, considere fraude, com um erro de 49%.

Modelo com custo

Pode ser mais importante para quem investiga as fraudes um tipo de classificação errada que outra. No caso, classificar como fraude algo que não é (erros do Tipo I - falso positivo) é bem menos importante que não classificar como fraude uma fraude (erros Tipo II - falso negativo).
A alteração da função de custo uniforme é feita fornecendo ao `rpart` uma matriz de custo que tem os pesos das classificações incorretas.
A posição mi,j da matriz indica o custo de classificar i por j quando obviamente $i \quad j$. Portanto mi,i = 0.

```
treino <- data.frame(ClaimID=1:7,
  RearEnd  = c(TRUE, TRUE, FALSE, FALSE, FALSE, FALSE, FALSE),
  Whiplash = c(TRUE, TRUE,  TRUE,  TRUE,  TRUE, FALSE, FALSE),
  Fraud    = c(TRUE, TRUE,  TRUE, FALSE, FALSE, FALSE, FALSE) )
```

```
treino
```

```
tree.custo <- rpart(Fraud ~ RearEnd + Whiplash,
                    data=treino, method="class",
                    maxdepth=1, minsplit=2, minbucket=1)
```

```
rpart.plot(tree.custo, cex=1.3)
```

O modelo identificou corretamente `RearEnd` como o critério de decisão. Porém um caso de fraude foi classificada incorretamente como não fraude. Para triplicar o custo de uma fraude classificada como fraude é preciso construir uma matriz de custo com os pesos.

```r
predito <- predict(tree.custo, treino)
```

```r
factor(predito[,1] > 0.5)
```

```r
treino$Fraud
```

```r
confusionMatrix(reference = factor(treino$Fraud), data = factor(predito[,1]>0.5))
```

```r
predito <- predict(tree.custo, treino)
```

```r
treino$Fraud
```

```r
predito
```

```r
confusionMatrix(reference = factor(treino$Fraud), data=factor(predito[,2]>0.5))
```

```r
custo <- matrix(c(0,1,3,0), byrow=TRUE, nrow=2)
print(custo)
```

	fraude	não fraude	explicação
predito como fraude	0	1	penalidade é `1` não ser fraude e dizer que é
predito como não fraude	3	0	penalidade é `3` ser fraude mas dizer que não é

```r
tree.custo <- rpart(Fraud ~ RearEnd + Whiplash, data=treino,
                    method="class",
                    maxdepth=1, minsplit=2, minbucket=1,
                    parms=list(loss=custo) )
```

```r
rpart.plot(tree.custo, cex=1.3)
```

Agora a variável preditora passou a ser `Wiplash`, acusando duas não fraudes como fraudes mas corretamente identificando todas fraudes.

Modelo com fator

É possível modelar utilizando uma variável com fatores. Aqui tem um exemplo com o estado de atividade do segurado.

```r
treino <- data.frame(
    ClaimID=c(1,2,3,4,5),
    Activity=factor(x=c("ativo", "ativo+", "ativo+", "inativo", "inativo-"),
                    levels = c("inativo-", "inativo", "ativo", "ativo+")),
    Fraud=c(FALSE, TRUE, TRUE, FALSE, TRUE) )
```

```r
print(treino)
```

```
tree.fator <- rpart(Fraud ~ Activity, data=treino,
                    method="class",
                    minsplit=2, minbucket=1)
```

```
rpart.plot(tree.fator, cex=1.3)
```

Os fatores não foram definidos como ordenados (inativo-, inativo, ativo, ativo+) e por isso `rpart` testou todas possibilidades de combinação de classes. Seria interessante que a ordem fosse levada em consideração pois é uma informação que ajuda a interpretar o modelo.

```
treino <- data.frame(
    ClaimID=c(1,2,3,4,5),
    Activity=factor(x=c("ativo", "ativo+", "ativo+", "inativo", "inativo-"),
                    levels = c("inativo-", "inativo", "ativo", "ativo+"),
                    ordered=TRUE),
    Fraud=c(FALSE, TRUE, TRUE, FALSE, TRUE) )
```

```
tree.fator <- rpart(Fraud ~ Activity, data=treino, method="class", minsplit=2, minbucket=1)
```

```
rpart.plot(tree.fator, cex=1.3)
```

Modelo com poda

Árvores de decisão completas tendem a sofrer de *overfiting* pois se esforçam em ser o mais adequadas possível aos dados de treinamento, perdendo generalização. Uma solução é a poda da árvore, garantindo que não fique muito complexa ao utilizar só as principais decisões.

```
treino <- data.frame(ClaimID=1:10,
  RearEnd  = c(TRUE, TRUE, TRUE, FALSE, FALSE, FALSE, FALSE,  TRUE,  TRUE, FALSE),
  Whiplash = c(TRUE, TRUE, TRUE,  TRUE,  TRUE, FALSE, FALSE, FALSE, FALSE,  TRUE),
  Activity = factor(x=c("ativo", "ativo+", "ativo+", "inativo", "inativo-",
                        "inativo", "inativo-", "ativo", "ativo", "ativo+"),
                    levels = c("inativo-", "inativo", "ativo", "ativo+"),
                    ordered=TRUE),
  Fraud = c(FALSE, TRUE, TRUE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, TRUE) )
```

```
print(treino)
```

Modelo com a árvore completa:

```
tree.poda <- rpart(Fraud ~ RearEnd + Whiplash + Activity, data=treino, method="class",
                   minsplit=2, minbucket=1, cp=-1)
```

```
rpart.plot(tree.poda, cex=1.3)
```

A árvore corretamente classifica todos as casos. Poderia, porém, ser o caso de a árvore ser *construída* para esses casos. Pode-se ver a importância de cada variável na construção da árvore com `variable.importance`:

```
print(tree.poda$variable.importance)
```

```
printcp(tree.poda)
```

O `rel error` de cada nível da árvore é a fração de observações mal classificadas no nível com relação à quantidade de observações em classes incorretas na raiz. Neste caso, com 10 observações e 5 fraudes, a raiz considerou classificar todos como fraude, acarretando um erro de 50%. A primeira divisão (é `ativo+`) gerou dois nós, um classificando como "fraude", com 3 casos de fraude (3 em 10 do total, todos 3 fraude) e outro como "não fraude" com 7 observações (7 em 10, 2 delas incorretas). A taxa de erros passou de 50% (5 erradas em 10) para 20% (2 erradas em 10), causando uma melhoria de 60% (com relação à raiz) em que há 2 erradas em 5 (40% do erro) ou 60% corretas (3 corretas das 5 erradas).

As taxas de erro da validação cruzada (*cross validation error rates*) e seus desvios padrão estão registrados na tabela como `xerror` e `xstd`, respectivamente.

Como regra, costuma-se podar uma árvore de decisão no `CP` da menor árvore que está dentro de um desvio padrão da árvore com o menor `xerror`. Neste exemplo, o menor `xerror` é 0,40, com um desvio padrão de 0,253. Procuramos então a menor árvore com `xerror` menor que (0,4 + 0,253) 0,653. Temos então o nível 2, com `CP` = 0,2. Podamos então a árvore com um `CP` (Parâmetro de Complexidade) ligeiramente maior que 0,2.

```
tree.podada <- prune(tree.poda, cp=0.21)
```

```
rpart.plot(tree.podada, cex=1.3)
```

Podemos usar a árvore de decisão podada para predizer propostas fraudulentas em um conjunto de dados desconhecidos.

```
teste <- data.frame(
    ClaimID=1:10,
    RearEnd  = c(FALSE, TRUE, TRUE, FALSE, FALSE, FALSE, FALSE,  TRUE,  TRUE, FALSE),
    Whiplash = c(FALSE, TRUE, TRUE,  TRUE,  TRUE, FALSE, FALSE, FALSE, FALSE,  TRUE),
    Activity = factor(
        x=c("inativo", "ativo+", "ativo+", "inativo", "inativo-",
            "inativo", "inativo-", "ativo", "ativo", "ativo+"),
    levels=c("inativo-", "inativo", "ativo", "ativo+"),
    ordered=TRUE) )
```

```
teste$FraudClass <- predict(tree.podada, newdata=teste, type="class")
teste$FraudProb  <- predict(tree.podada, newdata=teste, type="prob")
```

```
teste
```

Caso 3 - Banco

Referência

Base de árvore de decisão

Bibliotecas

```
rm(list=ls())
```

```
library(rpart)  # árvores de decisão
library(repr)   # representações (ajuste gráfico do jupyter)
library(caret)  # matriz de confusão
library(rpart.plot)
library(RColorBrewer)
```

```
grafdim <- function(x=11, y=9) { options(repr.plot.width=x, repr.plot.height=y) }
grafdim()
```

Preparação do ambiente

```
set.seed(997)
```

Dados originais

O banco de dados utilizado está em `bank.Zip` no repositório em Bank+Marketing.

```
orig <- read.csv(unzip("/home/max/ref/d/dataset/bank_marketing.zip","bank-full.csv"),
                 sep=";", header=TRUE)
file.remove("bank-full.csv")
```

```
head(orig)
```

```
str(orig)
```

```
print(summary(factor(orig$job)))
```

```
orig$job <- factor(orig$job)
```

```
print(summary(factor(orig$contact)))
```

```
orig$contact <- factor(orig$contact)
```

```
orig$contrata <- factor(orig$y)
```

```
orig$y <- NULL
```

```
print(summary(factor(orig$marital)))
```

```
orig$marital <- factor(orig$marital)
```

```
print(summary(factor(orig$education)))
```

```
orig$education <- factor(orig$education,
                         levels=c("unknown", "primary", "secondary", "tertiary"),
                         ordered=TRUE)
```

```
orig$housing <- factor(orig$housing)
```

```
orig$default <- factor(orig$default)
```

```
orig$loan    <- factor(orig$loan)
```

```r
x <- orig$month
x[x == "jan"] <- 1
x[x == "feb"] <- 2
x[x == "mar"] <- 3
x[x == "apr"] <- 4
x[x == "may"] <- 5
x[x == "jun"] <- 6
x[x == "jul"] <- 7
x[x == "aug"] <- 8
x[x == "sep"] <- 9
x[x == "oct"] <- 10
x[x == "nov"] <- 11
x[x == "dec"] <- 12
```

```r
x <- as.integer(x)
```

```r
x <- factor(x)
```

```r
orig$month <- x
```

```r
orig$poutcome <- factor(orig$poutcome)
```

```r
summary(orig)
```

```r
bank <- orig
```

```r
grafdim(12,6)
boxplot(bank$balance ~ bank$contrata, col="moccasin", horizontal=TRUE)
```

```r
summary(bank$balance[bank$balance < 0])
boxplot(bank$balance[bank$balance < 0] ~ bank$contrata[bank$balance < 0],
        col="moccasin", horizontal=TRUE)
```

```r
summary(bank)
```

```r
save(data="bank", file="bank.RData")
```

```r
ls()
```

Dados processados

```r
load("bank.RData")
```

```r
ls()
```

```r
summary(bank)
```

Separação dos conjuntos de *treino*, *validação* e *teste*

```
(num_obs    <- nrow(bank))
tx_teste    <- 0.15
tx_valida   <- 0.15
(num_teste  <- trunc(num_obs * tx_teste))
(num_valida <- trunc(num_obs * tx_valida))
(num_treino <- num_obs - num_teste - num_valida)
```

```
set.seed(997)
```

```
bank_treino <- bank
amostra  <- sample(1:num_obs, num_teste, replace=FALSE)
bank_teste  <- bank_treino[ amostra, ]
bank_treino <- bank_treino[-amostra, ]
amostra  <- sample(1:(num_obs - num_teste), num_valida, replace=FALSE)
bank_valida <- bank_treino[ amostra, ]
bank_treino <- bank_treino[-amostra, ]
head(bank_treino)
head(bank_valida)
```

A proporção de positivos deve ser próxima nos três conjuntos.

```
(sum(bank_treino$contrata == "yes") / nrow(bank_treino))
(sum(bank_valida$contrata == "yes") / nrow(bank_valida))
(sum(bank_teste$contrata == "yes")  / nrow(bank_teste))
```

A proporção de positivos ficou próxima de 12% nos três conjuntos, não há indício de favorecimento em algum conjunto. Porém a classe de estudo é bastante desbalanceada; alguma técnica de balanceamento deve ser utilizada.

Dados separados em *treino*, *validação* e *teste*

```
save(bank_treino, bank_valida, bank_teste, file="bank_treino_valida_teste.RData")
```

```
load("bank_treino_valida_teste.RData")
ls()
```

Modelo base de comparação (*baseline*)

```
summary(bank_treino)
```

```
grafdim(12,8)
hist(bank_treino$balance, breaks=100, col="moccasin")
```

```
hist(bank_treino$balance[bank_treino$balance < 0], breaks=100, col="moccasin")
```

```
(min_balance <-  min(c(bank_treino$balance, bank_teste$balance, bank_valida$balance) - 1))
```

```
bal_treino <- log(bank_treino$balance - min_balance) - log(-min_balance)
bal_teste  <- log(bank_teste$balance  - min_balance) - log(-min_balance)
bal_valida <- log(bank_valida$balance - min_balance) - log(-min_balance)
```

```r
# dt <- data.frame(bal_treino, rep(length(bal_treino), "treino"))
boxplot(balance ~ grupo, data.frame(balance=c(bal_treino, bal_valida, bal_teste),
                grupo=c(rep("treino",length(bal_treino)), rep("valida",length(bal_valida)),
                  rep("teste",length(bal_teste)))), horizontal=TRUE, col="moccasin")
```

```r
hist(bal_treino, breaks=100, col="moccasin")
```

```r
bank_treino$balance <- bal_treino
bank_valida$balance <- bal_valida
bank_teste$balance  <- bal_teste
```

```r
modelo_base <- rpart(contrata ~ . , data=bank_treino,
                     method="class",
                     minsplit=20, minbucket=7, maxdepth=30)
```

```r
printcp(modelo_base)
```

```r
print(modelo_base$variable.importance)
```

```r
grafdim(10,9)
plot(modelo_base)
text(modelo_base, cex=1.3, xpd=TRUE)
```

```r
grafdim(12,11)
rpart.plot(modelo_base, cex=1.1)
```

```r
predito_base <- predict(modelo_base, bank_valida, type="class")
```

```r
print(head(predito_base,12))
```

```r
table(predito_base, classe=bank_valida$contrata)
```

```r
(matconf_base <- confusionMatrix(predito_base, bank_valida$contrata, positive="yes"))
```

```r
(accbal_base <- matconf_base$byClass["Balanced Accuracy"])
```

```r
tabela <- as.data.frame(matconf_base$table)
tabela$Reference <- factor(tabela$Reference, levels=rev(levels(tabela$Reference)))
tabela
```

```r
grafdim(5,4)
ggplot(tabela, aes(Prediction, Reference, fill=Freq)) +
        geom_tile() + geom_text(aes(label=Freq)) +
        scale_fill_gradient(low="white", high="#009194") +
        labs(x="Prediction", y="Reference")
```

Muito caso positivo perdido por falso negativo.

Modelo com ponderação

Construção do vetor com os pesos. No caso, o peso de um caso é a proporção de casos da outra classe.

```
pesos <- numeric(nrow(bank_treino))
(qtd_yes <- sum(bank_treino$contrata == "yes"))
(qtd_no  <- sum(bank_treino$contrata == "no"))
(qtd <- qtd_yes + qtd_no)
nrow(bank_treino)
(py   <- qtd_yes / qtd)
(pn   <- qtd_no  / qtd)
pesos[bank_treino$contrata == "yes"] <- pn # peso de "sim" é a proporção de "não" e vice-versa
pesos[bank_treino$contrata == "no"]  <- py
print(head(pesos))
```

```
modelo_peso <- rpart(contrata ~ . , data=bank_treino,
                     method="class",
                     weight=pesos,
                     minsplit=20, minbucket=7, maxdepth=30)
```

```
printcp(modelo_peso)
```

```
print(modelo_peso$variable.importance)
```

Como regra, costuma-se podar uma árvore de decisão no CP da menor árvore que está dentro de um desvio padrão da árvore com o menor xerror. Neste exemplo, o menor xerror é 0,40, com um desvio padrão de 0,253. Procuramos então a menor árvore com xerror menor que (0,4 + 0,253) 0,653. Temos então o nível 2, com CP = 0,2. Podamos então a árvore com um CP (Parâmetro de Complexidade) ligeiramente maior que 0,2.

```
grafdim(12,12)
plot(modelo_peso)
text(modelo_peso, cex=1.2, xpd=TRUE)
```

```
rpart.plot(modelo_peso, cex=1.1)
```

```
predito_peso <- predict(modelo_peso, bank_treino, type="class")
```

```
(matconf_peso <- confusionMatrix(predito_peso, bank_treino$contrata, positive="yes"))
```

```
(accbal_peso <- matconf_peso$byClass["Balanced Accuracy"])
```

```
tabela <- as.data.frame(matconf_peso$table)
tabela$Reference <- factor(tabela$Reference, levels=rev(levels(tabela$Reference)))
tabela
```

```
grafdim(5,4)
ggplot(tabela, aes(Prediction, Reference, fill=Freq)) +
        geom_tile() + geom_text(aes(label=Freq)) +
        scale_fill_gradient(low="white", high="#009194") +
        labs(x="Prediction", y="Reference")
```

Melhorou bastante a captura dos casos positivos, mas ainda há uma quantidade muito grande de falsos positivos que, para o caso em questão, é menos pior que falso negativo.

Para um exemplo com poda, veja Modelo com poda, do Caso 2.

```
printcp(modelo_peso)
```

Como regra, costuma-se podar uma árvore de decisão no CP da menor árvore que está dentro de um desvio padrão da árvore com o menor `xerror`. Neste exemplo, o menor xerror é 0,36841, com um desvio padrão de 0,0096282. Como 0,36841 + 0,0096292 = 0,3780392 que ainda é inferior ao nível anterior (0,38370), a poda é no nível 6 mesmo.

Validação

Vamos analisar como o modelo com peso se comporta com os dados de validação.

```
predito_valida <- predict(modelo_peso, bank_valida, type="class")
```

```
(matconf_valida <- confusionMatrix(predito_valida, bank_valida$contrata, positive="yes"))
```

```
(accbal_valida <- matconf_valida$byClass["Balanced Accuracy"])
```

```
tabela <- as.data.frame(matconf_valida$table)
tabela$Reference <- factor(tabela$Reference, levels=rev(levels(tabela$Reference)))
tabela
```

```
ggplot(tabela, aes(Prediction, Reference, fill=Freq)) +
        geom_tile() + geom_text(aes(label=Freq)) +
        scale_fill_gradient(low="white", high="#009194") +
        labs(x="Prediction", y="Reference")
```

O desempenho do modelo para os dados de validação foi similar àquele dos dados de treino, com acurácia balanceada de 0,826 (comparada à de 8,828 do conjunto de treino).

Teste

Vamos examinar como o modelo se comporta com o conjunto de teste.

```
predito_teste <- predict(modelo_peso, bank_teste, type="class")
```

```
(matconf_teste <- confusionMatrix(predito_teste, bank_teste$contrata, positive="yes"))
```

```
(accbal_teste <- matconf_teste$byClass["Balanced Accuracy"])
```

```
tabela <- as.data.frame(matconf_teste$table)
tabela$Reference <- factor(tabela$Reference, levels=rev(levels(tabela$Reference)))
tabela
```

```
ggplot(tabela, aes(Prediction, Reference, fill=Freq)) +
        geom_tile() + geom_text(aes(label=Freq)) +
        scale_fill_gradient(low="white", high="#009194") +
        labs(x="Prediction", y="Reference")
```

O desempenho do modelo para os dados de teste foi similar àquele dos dados de treino e validação, com acurácia balanceada de 0,830 (comparada à de 8,828 do conjunto de treino e 0,826 do conjunto de validação). O modelo é razoavelmente bom e foi capaz de generalizar a predição mesmo para casos desconhecidos, não utilizados no treinamento.